

Massively Parallel Domain Decomposition Preconditioner on a GPU: Efficient Implementation and Fine-Tuning

✉ Mieszko Grodzicki

Institute of Applied Mathematics and Mechanics
University of Warsaw
m.grodzicki@mimuw.edu.pl

Abstract

Exploiting GPU processing power for numerical computation requires regular data structures and algorithms with a high degree of parallelism. In the context of PDE solvers, multigrid methods have been successfully adapted to meet these demands, whereas domain decomposition methods remain relatively unexplored on GPUs. In this paper, we develop and analyze a PyTorch-based GPU implementation of a massively parallel domain decomposition preconditioner. The proposed solver takes a sparse matrix as input, enabling straightforward handling of variable or high-contrast coefficients and non-uniform meshes.

To overcome bottlenecks in PyTorch (e.g., in block sparse matrix-vector products), we design custom CUDA kernels that attain 70–90% of the measured peak memory bandwidth of an NVIDIA A100 GPU. To further enhance performance, we leverage mixed-precision computation and fine-tune the preconditioner’s parameters, such as subdomain and coarse problem sizes. The resulting solver outperforms NVIDIA’s AmgX preconditioner in all tested scenarios, with a maximum speedup of $4.92\times$ observed in 3D for higher-order elements. Moreover, its memory footprint is relatively low: systems with up to 100M DoFs in 2D and 50M DoFs in 3D can be solved on a single A100 with 80GB of memory.

Keywords: domain decomposition · GPU · PyTorch · mixed precision · elliptic PDEs · additive Schwarz method · hybrid Schwarz method

1 Introduction

Graphics Processing Units (GPUs) play a crucial role in modern scientific computing. Their architecture, optimized for executing thousands of threads in parallel, offers instruction throughput orders of magnitude higher than that of CPUs. However, fully exploiting this potential requires algorithms capable of exposing massive, fine-grained parallelism [19].

This poses a challenge when employing GPUs to solve Partial Differential Equations (PDEs). The discretization of these problems typically yields large sparse linear systems that require iterative solvers like Conjugate Gradient (CG). Because the convergence rate of these solvers degrades as the system’s condition number scales with the problem size, effective preconditioning is essential. Much work has been devoted to adapting general-purpose preconditioners to GPUs. Successful examples include incomplete LU (ILU) [8, 20, 25] and algebraic multigrid (AMG) [6, 7, 16, 17]. However, because such black-box methods usually do not exploit the structure of the underlying PDE, problem-specific approaches can often deliver superior performance.

A broad class of such tailored preconditioners comprises Domain Decomposition Methods (DDMs) [23], which partition the computational domain into smaller subdomains and solve local problems independently, naturally enabling parallel execution. However, efficient GPU implementation of DDMs remains relatively unexplored. Existing efforts primarily focus on hybrid CPU-GPU implementations [22, 24, 26] and typically employ coarse-grained parallelism involving tens [24, 26] or at most thousands [22] of large subdomains. Since GPUs are designed for massive concurrency, the performance of these approaches depends heavily on parallelizing individual subdomain solves.

Here, we investigate a different strategy, designing a GPU-native DDM solver that leverages fine-grained parallelism to solve millions of small problems concurrently, naturally aligning with GPU architectures. Our model problem arises from the discretization of elliptic PDEs of the form

$$\begin{aligned} -\operatorname{div}(\rho \nabla u) &= f && \text{in } \Omega, \\ u &= 0 && \text{on } \partial\Omega, \end{aligned} \quad (1.1)$$

which occur, for example, in steady-state heat conduction [13]. Here f is a prescribed source term, ρ is a given medium property (e.g. conductivity) that may have jumps in heterogeneous media, and u is the unknown (e.g. temperature).

We consider a Discontinuous Galerkin discretization, which permits a nonoverlapping decomposition, therefore minimizing data dependencies. The resulting linear system is solved using the Preconditioned Conjugate Gradient (PCG) method with a massively parallel additive Schwarz preconditioner [11, 15]. In addition, we consider its hybrid variant [23] to increase the performance. The preconditioner's construction allows a controlled balance between parallel scalability and preconditioner quality, giving an opportunity to fine-tune the solver for efficiency.

To utilize highly optimized GPU primitives, our implementation relies primarily on PyTorch [1]. This choice is motivated by PyTorch's convenient high-level interface and strong efficiency across a range of GPU architectures. We assume that the linear system is explicitly assembled, an approach that prioritizes flexibility, accommodating variable coefficients and non-uniform meshes. This contrasts with specialized matrix-free approaches [9], which deliver superior performance but are usually restricted to specific problem classes.

The paper is organized as follows. [Section 2](#) briefly reviews the DG discretization of (1.1) to establish the model linear systems we aim to solve. [Section 3](#) defines the Schwarz preconditioners and establishes their algorithmic foundations, while [Section 4](#) details the GPU implementation and optimizations, focusing on custom CUDA kernels. Finally, [Section 5](#) evaluates the preconditioners across a range of parameters and demonstrates that our proposed solver outperforms NVIDIA's AmgX preconditioner [17] in all tested scenarios.

2 Discontinuous Galerkin discretization of the problem

Let $\Omega \subset \mathbb{R}^d$, where $d \in \{2, 3\}$, be an open, bounded polyhedral domain with Lipschitz boundary $\partial\Omega$. We assume that $f \in L^2(\Omega)$, $\rho \in L^\infty(\Omega)$ are prescribed, and that there exist positive constants c_0, c_1 such that $c_0 \leq \rho \leq c_1$ almost everywhere in Ω . The variational formulation of (1.1) is then: Find $u^* \in H_0^1(\Omega)$ such that

$$\int_{\Omega} \rho(x) \nabla u^*(x) \cdot \nabla v(x) \, dx = \int_{\Omega} f(x) v(x) \, dx \quad \forall v \in H_0^1(\Omega). \quad (2.1)$$

Owing to the boundedness of ρ , the existence and uniqueness of the solution is guaranteed by the Lax-Milgram lemma. The variational problem is discretized by a symmetric weighted interior penalty Discontinuous Galerkin (DG) method, which we briefly describe below.

Let $\{\mathcal{T}_h\}_{h>0}$ be a shape-regular family of affine triangulations of Ω . We fix one triangulation $\mathcal{T}_h$ (the fine mesh) and denote by $\mathcal{E}_h$ the set of all its faces (edges in 2D). For a given integer $p \geq 1$, we will approximate the solution of (2.1) in the broken finite element space

$$V_h = \{v \in L^2(\Omega) : v|_K \in \mathcal{P}_p \text{ for all } K \in \mathcal{T}_h\},$$

where $\mathcal{P}_p$ is the space of real polynomials of degree at most p . For $u, v \in V_h$, $K \in \mathcal{T}_h$ and $e \in \mathcal{E}_h$, we use the standard notation

$$(u, v)_K = \int_K u(x)v(x) dx, \quad \langle u, v \rangle_e = \int_e u(x)v(x) d\sigma_e(x),$$

with σ_e being the surface measure on e . The discrete variational problem then reads [11, 14]: Find $u_h^* \in V_h$ such that

$$\mathcal{A}_h(u_h^*, v_h) = (f, v_h)_{L^2(\Omega)} \quad \forall v_h \in V_h, \quad (2.2)$$

where $\mathcal{A}_h : V_h \times V_h \rightarrow \mathbb{R}$ is a bilinear form on V_h defined as

$$\mathcal{A}_h(u, v) = \sum_{K \in \mathcal{T}_h} (\rho \nabla u, \nabla v)_K + \sum_{e \in \mathcal{E}_h} \langle \gamma[u], [v] \rangle_e - \sum_{e \in \mathcal{E}_h} \langle [u], \{\rho \nabla v\}_\omega \rangle_e - \sum_{e \in \mathcal{E}_h} \langle \{\rho \nabla u\}_\omega, [v] \rangle_e.$$

For an interior face $e = \partial K_1 \cap \partial K_2$, $K_1, K_2 \in \mathcal{T}_h$, we set

$$\gamma = \frac{\delta p^2}{h_e} \frac{\rho_1 \rho_2}{\rho_1 + \rho_2}, \quad \omega_1 = \frac{\rho_2}{\rho_1 + \rho_2}, \quad \omega_2 = \frac{\rho_1}{\rho_1 + \rho_2},$$

where $\rho_i = \rho|_{K_i}$, $h_e = \text{diam}(e)$ and $\delta > 0$ is a prescribed constant. The weighted average and jump operators are then

$$\{\rho \nabla u\}_\omega = \omega_1 \rho_1 \nabla u_1 + \omega_2 \rho_2 \nabla u_2, \quad [u] = u_1 n_1 + u_2 n_2,$$

with $u_i = u|_{K_i}$, and n_i being the outward normal vector to K_i on e for $i = 1, 2$. On a boundary face $e \subseteq \partial K \cap \partial \Omega$ with the outward normal vector n , we take

$$\gamma = \frac{\delta p^2}{h_e} \rho, \quad \{\rho \nabla u\}_\omega = \rho \nabla u, \quad [u] = un.$$

Values of $\{\rho \nabla v\}$ and $[v]$ are defined analogously.

Let us fix a basis $\Phi = (\varphi_0, \dots, \varphi_{n-1})$ of the finite element space V_h . Elements of the stiffness matrix $A = [a_{ij}]$ are defined as $a_{ij} = \mathcal{A}_h(\varphi_i, \varphi_j)$ for $i, j = 0, \dots, n-1$. Equivalently, A is the matrix representation of the bilinear form $\mathcal{A}_h$ with respect to the basis Φ . Similarly, let us define elements of the load vector $b = (b_0, \dots, b_{n-1})$ as $b_i = (f, \varphi_i)_{L^2(\Omega)}$. The variational problem (2.2) is then equivalent to the linear system

$$Au^* = b, \quad (2.3)$$

where $u^* = (u_0, \dots, u_{n-1}) \in \mathbb{R}^n$ are the coefficients of u_h^* in the basis Φ , i.e. $u_h^* = \sum_{i=0}^{n-1} u_i \varphi_i$.

Theorem 2.1 ([14]) *There exists $\delta_0 > 0$ such that for all $h > 0$, $p \geq 1$ and $\delta \geq \delta_0$, the bilinear form $\mathcal{A}_h$ (and hence its corresponding matrix A) is symmetric and positive definite (SPD).*

From this point onward, we shall assume that $\delta \geq \delta_0$, and consequently, the conclusions of the above theorem hold.

3 Additive and hybrid Schwarz preconditioners

We aim to solve the linear system (2.3) using PCG, which requires an effective preconditioner for the matrix A . To this end, we employ the massively parallel nonoverlapping Schwarz Method based on [11], which we outline below.

We introduce two additional partitions of Ω : coarse mesh $\mathcal{T}_H = \{D_0, \dots, D_{N_H-1}\}$ and local solvers' mesh $\mathcal{T}_h = \{\Omega_0, \dots, \Omega_{N_H-1}\}$. We assume the inclusion

$$\mathcal{T}_H \subseteq \mathcal{T}_H \subseteq \mathcal{T}_h,$$

in the sense that $\mathcal{T}_H$ is a refinement of $\mathcal{T}_H$ and $\mathcal{T}_h$ is a refinement of $\mathcal{T}_H$ (see Figure 5.1). The finite element space V_h , hereafter denoted simply by V , is then decomposed into the sum

$$V = V_C + \sum_{i=0}^{N_H-1} V_i,$$

where the coarse space is

$$V_C = \{v \in V : v|_{D_j} \in \mathcal{P}_0 \text{ for } j = 0, \dots, N_H - 1\}$$

and the local spaces are

$$V_i = \{v \in V : v|_{\Omega_j} = 0 \text{ for all } j \neq i\}, \quad i = 0, \dots, N_H - 1.$$

Based on the above decomposition, the coarse solve operator $T_C: V \rightarrow V_C$ is prescribed by the relation

$$\mathcal{A}_h(T_C u, v_C) = \mathcal{A}_h(u, v_C) \quad \forall v_C \in V_C,$$

and analogously the local solve operators $T_i: V \rightarrow V_i$ satisfy

$$\mathcal{A}_h(T_i u, v_i) = \mathcal{A}_h(u, v_i) \quad \forall v_i \in V_i$$

for $i = 0, \dots, N_H - 1$. We consider the preconditioned additive Schwarz operator $T_{add}: V \rightarrow V$, which takes the form

$$T_{add} = T_C + \sum_{i=0}^{N_H-1} T_i. \quad (3.1)$$

In addition to the fully additive Schwarz method [2, 11], we examine its hybrid variant $T_{hyb}: V \rightarrow V$, given by [23]:

$$T_{hyb} = T_C + (I - T_C) \sum_{i=0}^{N_H-1} T_i (I - T_C).$$

Unlike [11], we do not use an inexact bilinear form to define T_C and T_i in either preconditioner. While a simplified form can be beneficial in a matrix-free context where entries are evaluated on-the-fly, it yields no computational advantage if assembled in advance, as it shares the sparsity pattern of the original form $\mathcal{A}_h$. Furthermore, maintaining both matrices would double the memory footprint, significantly reducing the problem size that fits on a single GPU. However, we later introduce a different type of “inexact” solvers by reducing the floating-point precision used for the local and coarse problems.

3.1 Condition number estimates

For completeness, we cite here results which provide sufficient conditions for producing an effective preconditioner for (2.3). Below h , H and $\mathcal{H}$ are the maximum diameters of elements in the fine mesh T_h , the solvers' mesh T_H and the coarse mesh $T_{\mathcal{H}}$, respectively, and by $\lesssim$ we denote an inequality up to a constant independent of h , H , $\mathcal{H}$, p and jumps of the coefficient ρ .

Theorem 3.1 ([3]) *Assume $\mathcal{T}_H = \mathcal{T}_{\mathcal{H}}$, meshes $\mathcal{T}_h$ and $\mathcal{T}_H$ are quasi-uniform, and $\rho \equiv \text{const}$ a.e. in Ω . Then the operator T_{add} is SPD with respect to $\mathcal{A}_h$, and its condition number satisfies*

$$\kappa(T_{add}) \lesssim p^2 \frac{\mathcal{H}}{h}.$$

Theorem 3.2 ([11]) *Assume $p = 1$, the meshes $\mathcal{T}_h$, $\mathcal{T}_H$ and $\mathcal{T}_{\mathcal{H}}$ are quasi-uniform, satisfy certain regularity conditions and the variation of the coefficient ρ is uniformly controlled independently of h , H and $\mathcal{H}$ (i.e. conditions (A1), (A2) in [11]). Then the operator T_{add} is SPD with respect to $\mathcal{A}_h$, and*

$$\kappa(T_{add}) \lesssim \frac{\mathcal{H}^2}{hH}.$$

Lemma 3.3 ([23]) *Assume that T_{add} is SPD with respect to $\mathcal{A}_h$. Then T_{hyb} is also SPD with respect to $\mathcal{A}_h$, and*

$$\kappa(T_{hyb}) \leq \kappa(T_{add}).$$

3.2 Matrix form of the preconditioners

Let R_C^T denote the matrix of the identity embedding $V_C \hookrightarrow V$ and let R_C be its transpose. Matrices R_i^T and R_i are defined analogously for the local spaces V_i , $i = 0, \dots, N_H - 1$. We define the coarse solver matrix as

$$A_C = R_C A R_C^T, \quad (3.2)$$

and the local solvers' matrices as

$$A_i = R_i A R_i^T, \quad i = 0, \dots, N_H - 1. \quad (3.3)$$

A standard calculation shows [23] that the matrix of the operator T_{add} might be expressed as $M_{add}A$, where

$$M_{add} = R_C^T A_C^{-1} R_C + \sum_{i=0}^{N_H-1} R_i^T A_i^{-1} R_i, \quad (3.4)$$

is the matrix of the additive Schwarz preconditioner for the system (2.3). Similarly, the matrix of the operator T_{hyb} can be expressed as $M_{hyb}A$, where

$$M_{hyb} = R_C^T A_C^{-1} R_C + M_{hyb}^{loc}$$

and

$$M_{hyb}^{loc} = \left(I - R_C^T A_C^{-1} R_C A \right) \sum_{i=0}^{N_H-1} R_i^T A_i^{-1} R_i \left(I - A R_C^T A_C^{-1} R_C \right). \quad (3.5)$$

In contrast to the additive case, we do not solve the preconditioned system

$$M_{hyb} A u = M_{hyb} b$$

directly using PCG. Instead, we employ the projected conjugate gradient algorithm [23] and equivalently solve the projected system

$$M_{hyb}^{loc}Av = M_{hyb}^{loc}b, \quad (3.6)$$

for $v \in \text{range}(I - R_C^T A_C^{-1} R_C A)$. The solution of (2.3) is then recovered as

$$u^* = R_C^T A_C^{-1} R_C b + v.$$

3.3 Algorithmic realization

The preconditioning matrices M_{add} and M_{hyb}^{loc} defined in (3.4) and (3.5), respectively, are used in the PCG method only to compute matrix-vector products with the residual. In this section, we sketch generic parallel algorithms to compute these products (which we later specialize for the GPU architecture in Section 4). To leverage the efficiency of PyTorch tensor operations, we impose an additional mild requirement on the local spaces V_i .

Assumption 3.4 *The number of degrees of freedom (DoFs) in each local space V_i is the same; that is $\dim V_i = k$ for some fixed integer k and all $i = 0, \dots, N_H - 1$.*

Under this assumption, all local solvers' matrices A_i have identical dimensions $k \times k$, which fits well with the Single-Instruction Multiple-Threads (SIMT) model of GPU computation [19]. Let us stress though, that the matrices A_i need not be identical, preserving flexibility in the mesh design and allowing for a variable coefficient ρ .

Let l_i be the number of local solvers' subdomains Ω_j that are contained in the element D_i of the coarse mesh $\mathcal{T}_H$, i.e.

$$l_i = |\{j : \Omega_j \subseteq D_i\}|, \quad i = 0, \dots, N_H - 1.$$

Note that we allow different values of l_i for different i . To shorten the notation, let us set

$$\bar{l}_0 = 0; \quad \bar{l}_i = l_1 + \dots + l_{i-1}, \quad i = 1, \dots, N_H.$$

Observe that $\bar{l}_{N_H} = N_H$.

Now, as the basis $\Phi = (\varphi_0, \dots, \varphi_{n-1})$ of V let us choose the standard finite element nodal basis. After an appropriate permutation of the indices, we may additionally assume that

$$\begin{aligned} \text{supp } \varphi_j &\subseteq \bar{\Omega}_i \iff ki \leq j < k(i+1), \\ \Omega_j &\subseteq D_i \iff \bar{l}_i \leq j < \bar{l}_{i+1}, \end{aligned}$$

which implies

$$\text{supp } \varphi_j \subseteq \bar{D}_i \iff k\bar{l}_i \leq j < k\bar{l}_{i+1}.$$

Then we may choose

$$\Phi_i = (\varphi_j : ki \leq j < k(i+1))$$

as the basis of V_i for $i = 0, \dots, N_H - 1$ and

$$\Phi_C = (\varphi_{k\bar{l}_i} + \dots + \varphi_{k\bar{l}_{i+1}-1} : i = 0, \dots, N_H - 1)$$

as the basis of V_C . As a result, for $i = 0, \dots, N_H - 1$, the matrix R_i has the following, very simple block structure:

$$R_i = \left[\underbrace{\begin{bmatrix} 0_k & 0_k & \dots & 0_k \end{bmatrix}}_{i \text{ blocks}} \mid \underbrace{\begin{bmatrix} I_k & 0_k & 0_k & \dots & 0_k \end{bmatrix}}_{(N_H - i - 1) \text{ blocks}} \right], \quad (3.7)$$

where I_k denotes the $k \times k$ identity matrix and 0_k the zero matrix of the same size. For the coarse space we have

$$R_C = \begin{bmatrix} \underbrace{1 \ 1 \ \dots \ 1}_{kl_0} & & & \\ & \underbrace{1 \ 1 \ \dots \ 1}_{kl_1} & & \\ & & \ddots & \\ & & & \underbrace{1 \ 1 \ \dots \ 1}_{kl_{N_H-1}} \end{bmatrix}. \quad (3.8)$$

Now from (3.3) and (3.7) we see that if $A = [a_{rs}]_{r,s=0,\dots,n-1}$, then

$$A_i = [a_{rs}]_{r,s=ki,\dots,k(i+1)-1}, \quad i = 0, \dots, N_H - 1,$$

i.e. A_i is a submatrix of A containing only the rows and columns indexed by $ki, \dots, k(i+1) - 1$. On the other hand, from (3.2) and (3.8) we obtain

$$A_C = [c_{ij}]_{i,j=0,\dots,N_H-1}, \quad \text{where} \quad c_{ij} = \sum_{r=k\bar{l}_i}^{k\bar{l}_{i+1}-1} \sum_{s=k\bar{l}_j}^{k\bar{l}_{j+1}-1} a_{rs}.$$

One final algorithmic detail remains. In the implementation of M_{hyb}^{loc} , we need to compute the action of A immediately followed by the restriction R_C . We can optimize this operation by precomputing the matrix $A_R = R_C A$. From (3.8) it follows that

$$A_R = [d_{ij}]_{i=0,\dots,N_H-1;j=0,\dots,n-1}, \quad \text{where} \quad d_{ij} = \sum_{s=k\bar{l}_i}^{k\bar{l}_{i+1}-1} a_{sj}.$$

We summarize the algorithmic realization of the Schwarz preconditioners in [Algorithms 3.1](#) to [3.3](#). Note that M_{hyb_loc} assumes that the input vector lies in the range of $I - AR_C^T A_C^{-1} R_C$. As shown in [\[23\]](#), this condition is naturally satisfied when the function is used within the PCG method applied to the projected system (3.6). Since $I - AR_C^T A_C^{-1} R_C$ is a projection (which can be verified by direct calculation), it need not be explicitly applied at the beginning of M_{hyb_loc} , thereby reducing computational cost.

4 Porting the method to the GPU

We chose NVIDIA GPUs with compute capability 8.0 or higher as the target architecture for our implementation, focusing on the A100 SXM4 with 80GB of memory for all benchmarks and experiments.

We adopted the PyTorch 2 framework [\[1\]](#) for GPU-accelerated linear algebra operations. It provides a convenient, high-level interface that enables a concise implementation, automates memory management, and facilitates portability across different hardware platforms. Moreover, on NVIDIA GPUs, PyTorch delegates most operations to fast, highly optimized NVIDIA libraries, restricting the need for custom kernels to the few cases discussed below.

4.1 PCG iteration

The outermost layer of the solver is the PCG method, implemented by leveraging the high-level tensor operations provided by PyTorch. Its cost is dominated by two key components: the application of the Schwarz preconditioner and the product with the system matrix A . The former is of our

Algorithm 3.1: Schwarz Preconditioners – Setup

Input: $A, k, l = (l_0, \dots, l_{N_H-1})$
Output: $\bar{l}$, COARSESOLVE, LOCALSOLVE[0 .. $N_H - 1$]

```

1  $\bar{l} \leftarrow \text{CumulativeSums}(l)$ 
2 parallel for  $i = 0, \dots, N_H - 1$  do  $\triangleright A_i \leftarrow R_i A R_i^T$ 
3    $A_i \leftarrow A[k_i \dots k(i+1) - 1][k_i \dots k(i+1) - 1]$ 
4    $\text{LOCALSOLVE}[i] \leftarrow \text{InitLocalSolver}(A_i)$ 
5 end
6 parallel for  $i, j = 0, \dots, N_H - 1$  do  $\triangleright A_C \leftarrow R_C A R_C^T$ 
7    $A_C[i][j] \leftarrow \text{Sum}(A[k_{\bar{l}_i} \dots k_{\bar{l}_{i+1}} - 1][k_{\bar{l}_j} \dots k_{\bar{l}_{j+1}} - 1])$ 
8 end
9  $\text{COARSESOLVE} \leftarrow \text{InitCoarseSolver}(A_C)$ 
10 parallel for  $i = 0, \dots, N_H - 1; j = 0, \dots, n - 1$  do  $\triangleright A_R \leftarrow R_C A$ 
11    $A_R[i][j] \leftarrow \text{Sum}(A[k_{\bar{l}_i} \dots k_{\bar{l}_{i+1}} - 1][j])$ 
12 end
```

Algorithm 3.2: Additive Schwarz preconditioner – application

Require: $k, l, \bar{l}$, COARSESOLVE, LOCALSOLVE[0 .. $N_H - 1$]

```

1 function M_add( $x$ )  $\triangleright M_{\text{add}}x$ 
2   parallel for  $i = 0, \dots, N_H - 1$  do
3      $x^{(i)} \leftarrow x[k_i \dots k(i+1) - 1]$   $\triangleright x^{(i)} \leftarrow R_i x$ 
4      $y^{(i)} \leftarrow \text{LOCALSOLVE}[i](x^{(i)})$   $\triangleright y^{(i)} \leftarrow A_i^{-1} x^{(i)}$ 
5      $y[k_i \dots k(i+1) - 1] \leftarrow y^{(i)}$   $\triangleright y \leftarrow R_0^T y^{(0)} + \dots + R_{N_H-1}^T y^{(N_H-1)}$ 
6   end
7   parallel for  $i = 0, \dots, N_H - 1$  do  $\triangleright x^C \leftarrow R_C x$ 
8      $x^C[i] \leftarrow \text{Sum}(x[k_{\bar{l}_i} \dots k_{\bar{l}_{i+1}} - 1])$ 
9   end
10   $y^C \leftarrow \text{COARSESOLVE}(x^C)$   $\triangleright y^C \leftarrow A_C^{-1} x^C$ 
11   $z \leftarrow \text{RepeatInterleave}(y^C, k \cdot l)$   $\triangleright z \leftarrow R_C^T y^C$ 
12  return  $y + z$ 
13 end
```

primary interest and is discussed in detail in [Section 4.2](#). Nevertheless, when the preconditioner is sufficiently lightweight, the cost of the matrix-vector product becomes significant. For this reason, we address its optimization as well, as described in [Section 4.3](#).

4.2 Schwarz preconditioners

Since the setup cost is amortized over many solves, our optimization efforts focus on the application of the preconditioners to a vector, heavily leveraging the high-level linear algebra operations provided by PyTorch. [Assumption 3.4](#) and the consecutive numbering of DoFs allow us to implement R_i and R_i^T as simple reshaping operations, requiring no data movement. Operators R_C and R_C^T are implemented using `torch.segment_reduce` and `torch.repeat_interleave`. Together, these realizations make the implementation of either preconditioner very concise. We illustrate this

Algorithm 3.3: Hybrid Schwarz preconditioner – application

Require: $k, l, \bar{l}$, COARSESOLVE, LOCALSOLVE[0.. $N_H - 1$], A_R

```

1 function M_hyb_loc( $x$ )                                 $\triangleright M_{hyb}^{loc}x$  for  $x \in \text{range}(I - AR_C^T A_C^{-1} R_C)$ 
2   parallel for  $i = 0, \dots, N_H - 1$  do
3      $x^{(i)} \leftarrow x[k i \dots k(i+1) - 1]$                  $\triangleright x^{(i)} \leftarrow R_i x$ 
4      $y^{(i)} \leftarrow \text{LOCALSOLVE}[i](x^{(i)})$                  $\triangleright y^{(i)} \leftarrow A_i^{-1} x^{(i)}$ 
5      $y[k i \dots k(i+1) - 1] \leftarrow y^{(i)}$                  $\triangleright y \leftarrow R_0^T y^{(0)} + \dots + R_{N_H-1}^T y^{(N_H-1)}$ 
6   end
7    $z \leftarrow \text{MatVec}(A_R, y)$                                  $\triangleright z \leftarrow R_C A y$ 
8    $w \leftarrow \text{COARSESOLVE}(z)$                              $\triangleright w \leftarrow A_C^{-1} z$ 
9    $t \leftarrow \text{RepeatInterleave}(w, k \cdot l)$                  $\triangleright t \leftarrow R_C^T w$ 
10  return  $y - t$ 
11 end

```

with the PyTorch implementation of the additive variant shown in Listing 4.1; the hybrid Schwarz preconditioner is implemented similarly. Additional performance gains are achieved by automatically fusing compatible operations into a single kernel using `torch.compile` [1].

Listing 4.1: PyTorch implementation of the additive Schwarz preconditioner

```

1  @torch.compile
2  def m_add(self, x: torch.Tensor) -> torch.Tensor:          #  $x \mapsto M_{add}x$ 
3    x_lower_precision = x.to(self.preconditioner_precision or x.dtype)
4    x_i = x_lower_precision.reshape(self.n_solvers, -1)      #  $x^{(i)} \leftarrow R_i x$ 
5    y_i = self.local_solvers.solve(x_i)                      #  $y^{(i)} \leftarrow A_i^{-1} x^{(i)}$ 
6    y = y_i.flatten()                                         #  $y \leftarrow \sum_i R_i^T y^{(i)}$ 
7    x_solvers = x_lower_precision.reshape(self.n_solvers, -1).sum(dim=1)
8    x_c = torch.segment_reduce(x_solvers, reduce="sum", offsets=self.solvers_per_coarse_scan) #  $x^C \leftarrow R_C x$ 
9    )
10   )
11   y_c = self.coarse_solver.solve(x_c)                        #  $y^C \leftarrow A_C^{-1} x^C$ 
12   y_solvers = y_c.repeat_interleave(
13     self.solvers_per_coarse, output_size=self.n_solvers
14   )
15   y += y_solvers.repeat_interleave(                          #  $y \leftarrow y + R_C^T y^C$ 
16     self.dofs_per_solver, output_size=y.shape[0]
17   )
18   return y.to(x.dtype)

```

For the coarse solver we employ the NVIDIA cuDSS library (v0.7.1) [21], a state-of-the-art direct sparse solver. Since the coarse matrix A_C is SPD, we call cuDSS with the option `CUDSS_MTYPE_SPD`, enabling sparse Cholesky factorization. The local solvers, conversely, are implemented from scratch to maximize performance in the context of Schwarz preconditioners, as described below.

4.2.1 Massively parallel local solvers

Since local solvers' matrices A_i will be typically small to maximize parallelism of the Schwarz preconditioners, it is profitable to use dense solvers for the local problems. However, for small

matrices, standard methods based on LU or Cholesky factorizations are difficult to parallelize. For this reason, we implement the local solves as multiplications by precomputed inverses A_i^{-1} . This approach promotes regular memory access patterns, yielding performance gains that outweigh the drawbacks of explicit inversion in this context. Following [4], we compute the inverses in high precision, but downcast them for the solution phase to accelerate the solver.

Given [Assumption 3.4](#), the inverses $A_0^{-1}, \dots, A_{N_H-1}^{-1}$ are computed with a single call to `torch.linalg.inv` and stored as a tensor of shape (N_H, k, k) . The solution phase thus reduces to a batched dense matrix-vector product $y^{(i)} = A_i^{-1}x^{(i)}$. Although PyTorch supports this operation via `cublas<t>gemvStridedBatched`, our experiments revealed that this cuBLAS function exhibits poor performance for small matrix sizes ($k < 64$). To address this limitation, we designed two custom algorithms implemented as CUDA kernels.

The first, denoted as BMV-I and shown in [Algorithm 4.2](#), implements a straightforward approach. Each thread computes a single element of some $y^{(i)}$ by alternately reading elements of the corresponding $x^{(i)}$ and a single row of A_i^{-1} (stored in the tensor `mat`). The second, denoted as BMV-S, first loads all elements of $x^{(i)}$ into shared memory so that, after synchronization, they can be reused by all threads computing $y^{(i)}$. In BMV-I, the assignment of threads to output elements is independent of the batch structure; in contrast, BMV-S requires that each $y^{(i)}$ is computed within a single thread block.

Algorithm 4.2: Batched dense matrix-vector product – BMV-I kernel

```

1  $t \leftarrow \text{blockDim}.x \cdot \text{blockIdx}.x + \text{threadIdx}.x$ 
2 if  $t \geq N_H k$  then return
3  $i \leftarrow \lfloor t/k \rfloor$ 
4  $r \leftarrow t \bmod k$ 
5  $\text{acc} \leftarrow 0$ 
6 for  $c \leftarrow 0$ ;  $c < k$ ;  $c++$  do
7    $\text{mat\_elem} \leftarrow \text{mat}[i, c, r]$ 
8    $\text{mat\_elem\_higher\_prec} \leftarrow \text{ConvertTo}(\text{mat\_elem}, \text{typeof}(x\_elem))$ 
9    $x\_elem \leftarrow x[i, c]$ 
10   $\text{acc} += \text{mat\_elem\_higher\_prec} \cdot x\_elem$ 
11 end
12  $y[i, r] \leftarrow \text{acc}$ 

```

The matrices A_i^{-1} are assumed to be stored in column-major order (cf. lines 7 and 12 in [Algorithms 4.2](#) and [4.3](#), respectively), which ensures that most accesses to their elements can be coalesced (see [19, Section 5.3.2]). If the matrices A_i^{-1} are stored in lower precision than that of $x^{(i)}$ and $y^{(i)}$, the upcasting of A_i^{-1} elements is performed immediately after fetching them from memory (lines 8 and 13, respectively), ensuring that multiplication and accumulation are always performed in the precision of $x^{(i)}$ and $y^{(i)}$. Because these are only two floating-point operations per element of A_i^{-1} , the algorithm is memory-bound and reducing computation precision does not improve performance, which our experiments confirm.

As is usual for memory-bound operations, we measure the performance in terms of memory throughput, calculated as the total size of the input and output tensors divided by the kernel execution time. Results are similar across all precision combinations; therefore, in [Figure 4.1](#) we present only two representative cases. BMV-S sustains high throughput across the tested matrix sizes, reaching over 84% of the achievable A100 memory bandwidth. It is outperformed by BMV-I only for a few of the smallest matrix sizes, where the overhead of synchronizing threads becomes

Algorithm 4.3: Batched dense matrix-vector product – BMV-S kernel

```

1  $batches\_per\_block \leftarrow \lfloor blockDim.x/k \rfloor$ 
2  $local\_batch\_idx \leftarrow \lfloor threadIdx.x/k \rfloor$ 
3 if  $local\_batch\_idx \geq batches\_per\_block$  then return
4  $i \leftarrow blockIdx.x \cdot batches\_per\_block + local\_batch\_idx$ 
5  $r \leftarrow threadIdx.x \bmod k$ 
6 if  $i \geq N_H$  then return
7  $shared\_x[0..blockDim.x-1] \leftarrow$  init shared array of the same type as  $x$ 
8  $shared\_x[threadIdx.x] \leftarrow x[i, r]$ 
9  $sync\_threads()$ 
10  $acc \leftarrow 0$ 
11 for  $c \leftarrow 0; c < k; c++$  do
12    $mat\_elem \leftarrow mat[i, c, r]$ 
13    $mat\_elem\_higher\_prec \leftarrow \text{ConvertTo}(mat\_elem, \text{typeof}(x\_elem))$ 
14    $x\_elem \leftarrow shared\_x[local\_batch\_idx \cdot k + c]$ 
15    $acc += mat\_elem\_higher\_prec \cdot x\_elem$ 
16 end
17  $y[i, r] \leftarrow acc$ 

```

significant. The performance of the PyTorch baseline is significantly worse, especially for the smallest matrices, where it is as much as 10–70 $\times$ slower than our algorithms. We therefore employ our custom kernels in the local solvers, using BMV-I for the smallest matrices ($k \leq 5$ or $k \leq 15$, depending on the precision) and BMV-S otherwise.

4.3 BSR matrix-vector product

To optimize the multiplication by the stiffness matrix A , we exploit its structure. After a suitable permutation, the DoFs associated with each element of $\mathcal{T}_h$ form a sequence of consecutive entries, allowing the matrix A to be represented in the Block Sparse Row (BSR) format [20, Section 3.3.6] with a block size b equal to the number of DoFs per element. Compared to CSR, the BSR format reduces the size of the row-offset array by a factor of b and the column-index array by a factor of b^2 . Because sparse matrix-vector multiplication is typically memory-bound, these savings translate directly into improved matrix-vector performance.

Multiple BSR matrix-vector algorithms are available. The simplest approach would be to use the `cusparsedbsrmv` routine from the cuSPARSE library, which is accessible via a PyTorch interface. However, in our benchmarks it underperformed the CSR matrix-vector product from the same library. Therefore, to improve performance, we implemented (in CUDA kernels) four algorithms from [12]: block-by-block, column-by-column, row-per-thread without synchronization (RPT) and row-per-thread with synchronization (RPT-S). We also introduced a modified RPT variant, RPT-I, which assigns rows to threads irrespective of thread block boundaries (see Algorithm 4.4). In the pseudocode, the BSR matrix A is represented by the arrays $A.crow_indices$, $A.col_indices$ and $A.values$. The blocks of $A.values$ are assumed to be stored in column-major order within each block (cf. line 10), which follows [12] and ensures coalesced memory accesses.

All algorithms were benchmarked on multiple 2D and 3D problems with varying polynomial degree p ; the results are shown in Figure 4.2. The `cusparselt>bbsrmv` routine, which we use as a baseline, is competitive only for $b = 4$ and $b = 15$. In the remaining cases it performs poorly, being

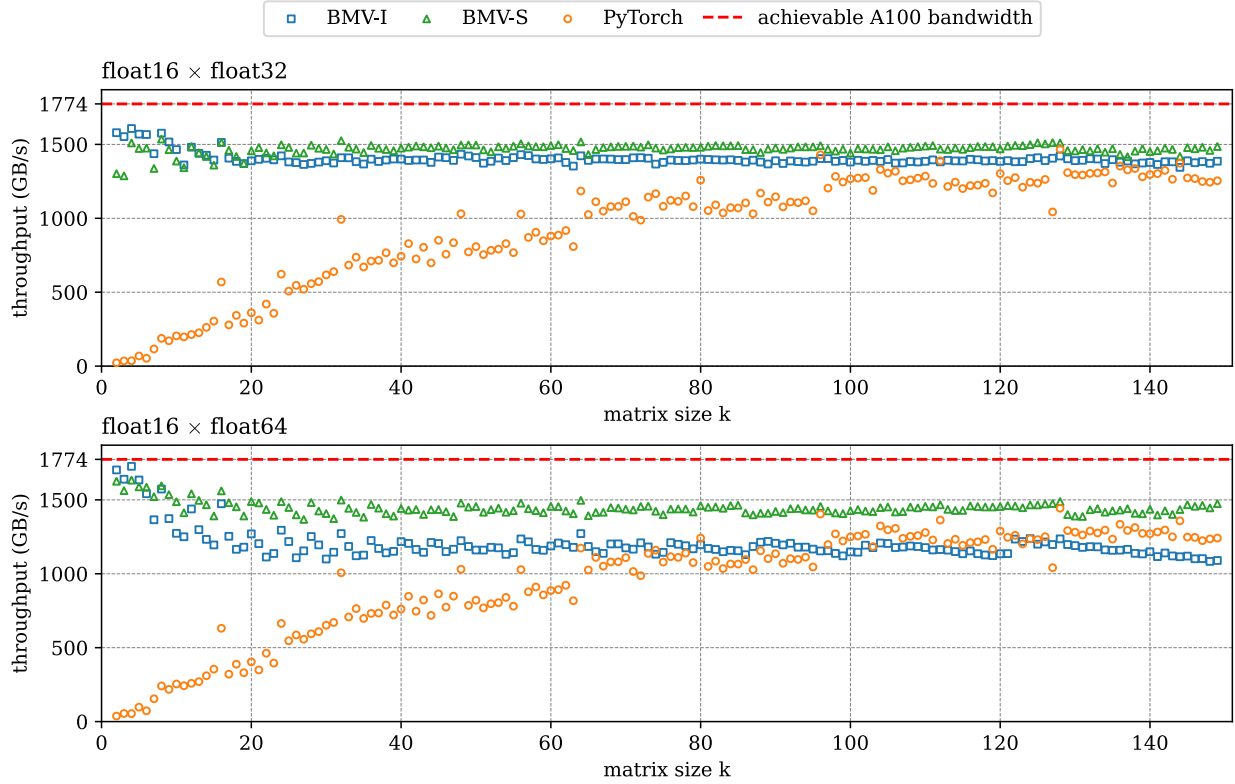


Figure 4.1. Performance of batched dense matrix-vector product: our implementations versus PyTorch. The red line indicates the peak memory bandwidth as measured by the `triad` benchmark from the BabelStream suite [10].

Algorithm 4.4: BSR matrix-vector product – RPT-I kernel

```

1 target_row  $\leftarrow$  blockDim.x · blockIdx.x + threadIdx.x
2 target_block_row  $\leftarrow$   $\lfloor$ target_row / b $\rfloor$ 
3 if target_block_row  $\geq$  A.crow_indices.size − 1 then return
4 r  $\leftarrow$  target_row mod b
5 first_block  $\leftarrow$  A.crow_indices[target_block_row]
6 after_last_block  $\leftarrow$  A.crow_indices[target_block_row + 1]
7 local_out  $\leftarrow$  0
8 for block  $\leftarrow$  first_block; block < after_last_block; block++ do
9   for c  $\leftarrow$  0; c < b; c++ do
10    A_elem  $\leftarrow$  A.values[block][c][r]
11    x_elem  $\leftarrow$  x[A.col_indices[block] · bs + c]
12    local_out += A_elem · x_elem
13   end
14 end
15 global_out[target_row]  $\leftarrow$  local_out

```

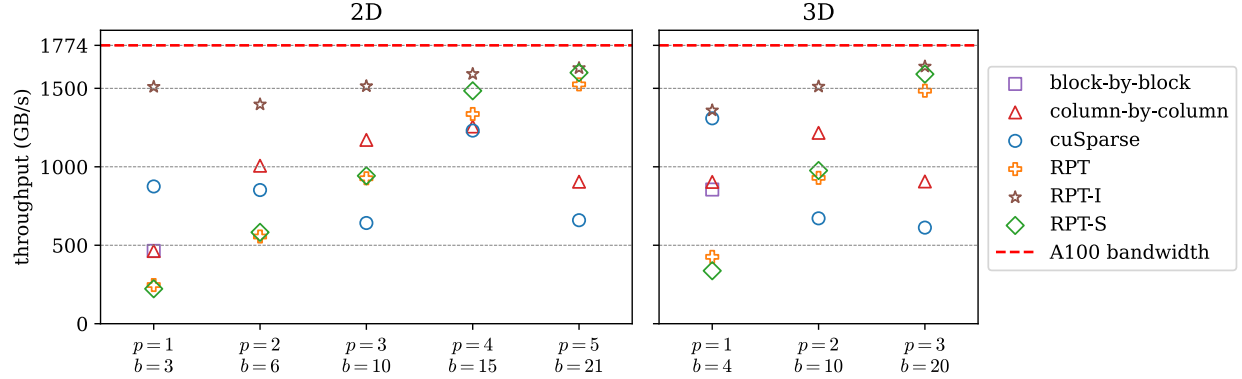


Figure 4.2. Performance of our implementations of the BSR matrix-vector product for various discretization parameters. The block-by-block algorithm requires $b \leq 5$. The red line indicates the peak memory bandwidth as measured by the `triad` benchmark from the BabelStream suite [10].

as much as $2.68\times$ slower than our implementations. We suspect that the cuSPARSE algorithm is optimized for block sizes being close to powers of two, and is therefore not well suited for our use case.

Relative performance of the algorithms introduced in [12] (column-by-column, block-by-block, RPT and RPT-S), agrees with the conclusions reported there. Absolute throughput numbers are substantially higher than in [12], as expected given the Kepler hardware used in that study versus much more advanced Ampere architecture of the A100 GPU employed here. Nevertheless, on the A100 all these algorithms are outperformed by the straightforward RPT-I approach, which is the fastest method for all tested scenarios. In addition, it attains very high memory throughput, ranging from 77% to 92% of the measured A100 peak bandwidth. Combined with the memory savings of the BSR format, this yields a $1.42\text{--}1.69\times$ speedup over the CSR matrix-vector product from cuSPARSE. For these reasons, we use the BSR format and the RPT-I algorithm as the default configuration in our PCG implementation.

5 Parameter tuning and performance evaluation

In this section, we discuss the fine-tuning of the additive and hybrid Schwarz preconditioners through a series of numerical experiments. We examine the impact of preconditioner parameters on efficiency and finally benchmark our solver against NVIDIA’s AmgX.

5.1 Experimental setup

In all experiments we consider $\Omega = (0, 1)^d$ with $d \in \{2, 3\}$ and fix $\rho(x) = 1 + \sum_{i=1}^d x_i^2$. We set $\delta = 7$ for $d = 2$ and $\delta = 17$ for $d = 3$. The right-hand side vector b is manufactured such that $u(x) = \prod_{i=1}^d \sin(4\pi x_i)$ is the exact solution of (1.1). Note that the experiments are conducted in the general three-grid scenario with $p \geq 1$, which is not covered by Theorems 3.1 and 3.2, but was partially addressed in [15]. In this setting we expect the condition number of the preconditioned operators to satisfy

$$\kappa(T_{hyb}) \leq \kappa(T_{add}) \lesssim p^2 \frac{\mathcal{H}^2}{hH}. \quad (5.1)$$

We introduce two types of uniform meshes for use in the experiments: composed of either hypercubes or simplices. For $d \in \{2, 3\}$ and an integer $m \geq 0$, let $\mathcal{C}_m^d$ denote the partition of

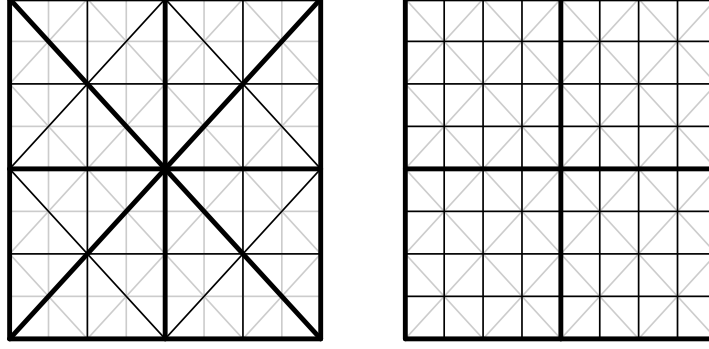


Figure 5.1. Example meshes and their refinement hierarchies for $d = 2$: $\mathcal{S}_1 \subseteq \mathcal{S}_2 \subseteq \mathcal{S}_3$ (left) and $\mathcal{C}_1 \subseteq \mathcal{C}_3 \subseteq \mathcal{S}_3$ (right).

Ω into 2^{md} equal hypercubes (squares in 2D, cubes in 3D) of side length 2^{-m} . The mesh $\mathcal{S}_m^d$ is obtained by subdividing each hypercube of $\mathcal{C}_m^d$ into simplices: via the natural diagonal subdivision into 2 triangles in 2D, and by the Coxeter-Freudenthal-Kuhn triangulation into 6 tetrahedra in 3D. When the dimension is clear from the context, we drop the superscript and write $\mathcal{C}_m$ and $\mathcal{S}_m$.

Examples for $d = 2$ are shown in Figure 5.1. In $\mathcal{S}_m^d$ (for both $d = 2, 3$) we use an alternating diagonal orientation across neighboring cells. Our experiments have shown that this alternating subdivision yields smaller approximation errors and faster convergence of all considered solvers than the non-alternating one. The alternation is chosen consistently for all m , so that $\mathcal{S}_m^d \subseteq \mathcal{S}_{m+1}^d$ for $m \geq 0$. Moreover, $\mathcal{C}_m^d \subseteq \mathcal{C}_{m+1}^d$ and $\mathcal{C}_m^d \subseteq \mathcal{S}_m^d$ for all $m \geq 0$. We always set $\mathcal{T}_h = \mathcal{S}_m$ and select $(\mathcal{T}_h, \mathcal{T}_H)$ from $\{(\mathcal{S}_M, \mathcal{S}_M), (\mathcal{C}_M, \mathcal{S}_M), (\mathcal{C}_M, \mathcal{C}_M)\}$ for some integers $0 \leq \mathcal{M} \leq M \leq m$. Although we utilize uniform meshes to easily compare different granularities of parallelism, our implementation supports arbitrary meshes, provided Assumption 3.4 is satisfied.

Mesh generation and discretization are performed in advance on a CPU using DOLFINx 0.9.0 [5]. The resulting stiffness matrix A and load vector b are transferred to the GPU, which then executes both the setup and the solution phase of the solver. While solutions are computed in double precision, the preconditioner may internally use lower-precision arithmetic, as described in Section 5.2.

All experiments are performed on an NVIDIA A100 SXM4 80GB. We use CUDA Events [19, Section 3.2.8.8] to measure GPU timings. The general measurement procedure is as follows. We perform N rounds, each consisting of K consecutive runs of the measured phase, and record the total time of each round independently. For each round, we compute the per-run time as the round time divided by K , and we report the minimum per-run time over all rounds. The parameters N and K are selected based on the expected duration of the measured phase.

Our primary metric in this section is the solution time of the system (2.3), defined as the time required to reduce the Euclidean norm of the initial residual by a factor of 10^9 using PCG. It excludes CPU-GPU data transfer and preconditioner setup, since these costs should be amortized over multiple solves with the same matrix A in typical production use.

5.2 Exploiting mixed precision to improve efficiency

We introduce two levels of precision control for both the additive and hybrid Schwarz preconditioners. First, the floating-point precision of all intermediate vectors computed within the preconditioner application can be set to either double (`fp64`) or single (`fp32`). Second, the inverses of the local solvers' matrices may be stored in one of four precisions: double (`fp64`), single (`fp32`), half (`fp16`)

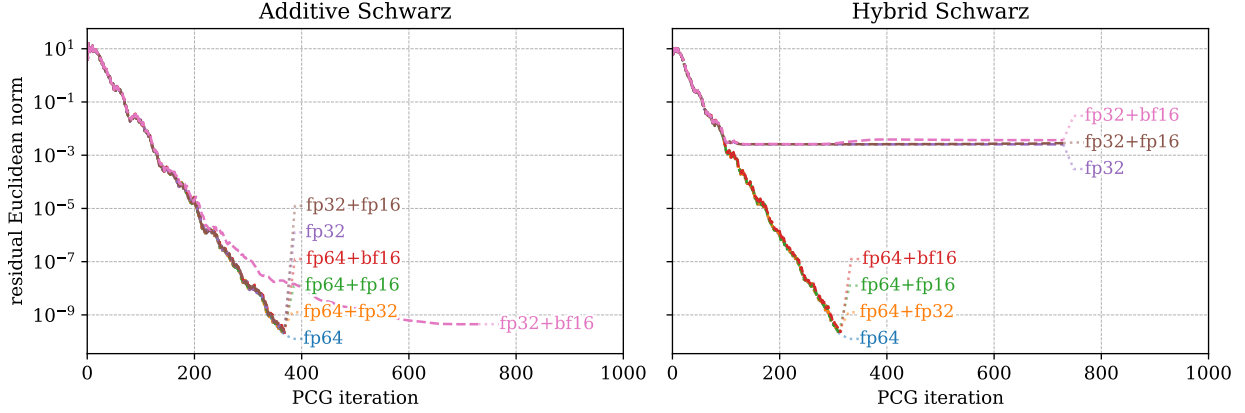


Figure 5.2. Euclidean norm of the residual at each PCG iteration for different preconditioner precision settings (selected case). Parameters: $d = 2$, $p = 5$, $\mathcal{T}_h = \mathcal{S}_8$ and $\mathcal{T}_H = \mathcal{T}_H = \mathcal{S}_6$.

or brain floating point (bf16); see Section 4.2.1. Since local solvers are not permitted to use higher precision than the preconditioner itself, this yields seven configurations. They are named after the main preconditioner precision; if the local solvers use a different type, it appears after a '+' sign. We test all configurations on three discrete systems, constructed with $d = 2$, $\mathcal{T}_h = \mathcal{S}_m$ and $(p, m) \in \{(1, 10), (3, 9), (5, 8)\}$.

5.2.1 Impact of precision settings on PCG convergence

Single-precision variants of the hybrid operator (fp32, fp32+fp16 and fp32+bf16) fail to achieve the desired residual reduction in all tested scenarios. The convergence behavior for the selected test case is shown in the right panel of Figure 5.2. While the double-precision variants converge steadily at the same rate, the single-precision runs stagnate at residual norms of approximately 10^{-3} . We suspect this is caused by elimination of the initial projection step in the hybrid preconditioner (see Section 3.3), which makes it more sensitive to precision reduction.

Another problematic configuration is the additive operator with fp32+bf16, which requires $2.5\times$ more iterations than fp64 in the extreme case $p = 5$, $m = 8$, $M = \mathcal{M} = 6$. A comparison of the convergence behavior for this scenario is shown in the left panel of Figure 5.2. While all other variants converge at similar rates, the fp32+bf16 run begins to slow when the residual norm reaches approximately 10^{-5} and eventually plateaus near 10^{-9} .

In contrast, the other mixed-precision configurations, including the most aggressive fp32+fp16 for the additive and fp64+fp16 for the hybrid preconditioner, show none or only minor increases in iteration counts compared to the double-precision baseline. We attribute this to the fact that half precision is used only to store the inverses of the local solvers' matrices, whereas their application to batches of vectors is performed in single precision.

5.2.2 Impact of precision settings on solver performance

The component-wise timings for all tested scenarios are presented in Figure 5.3. The most significant speedups relative to fp64 occur in the local solvers: $1.77\text{--}3.31\times$ for fp64+fp16 and $2.63\text{--}3.47\times$ for fp32+fp16, consistent with Section 4.2.1. As expected, +bf16 variants offer no performance advantage over +fp16; given the convergence issues they should be avoided.

By contrast, the time spent in the coarse CUDSS solver is only marginally affected, showing speedups of $1.07\text{--}1.18\times$ when using float32 instead of float64. This is expected since CUDSS

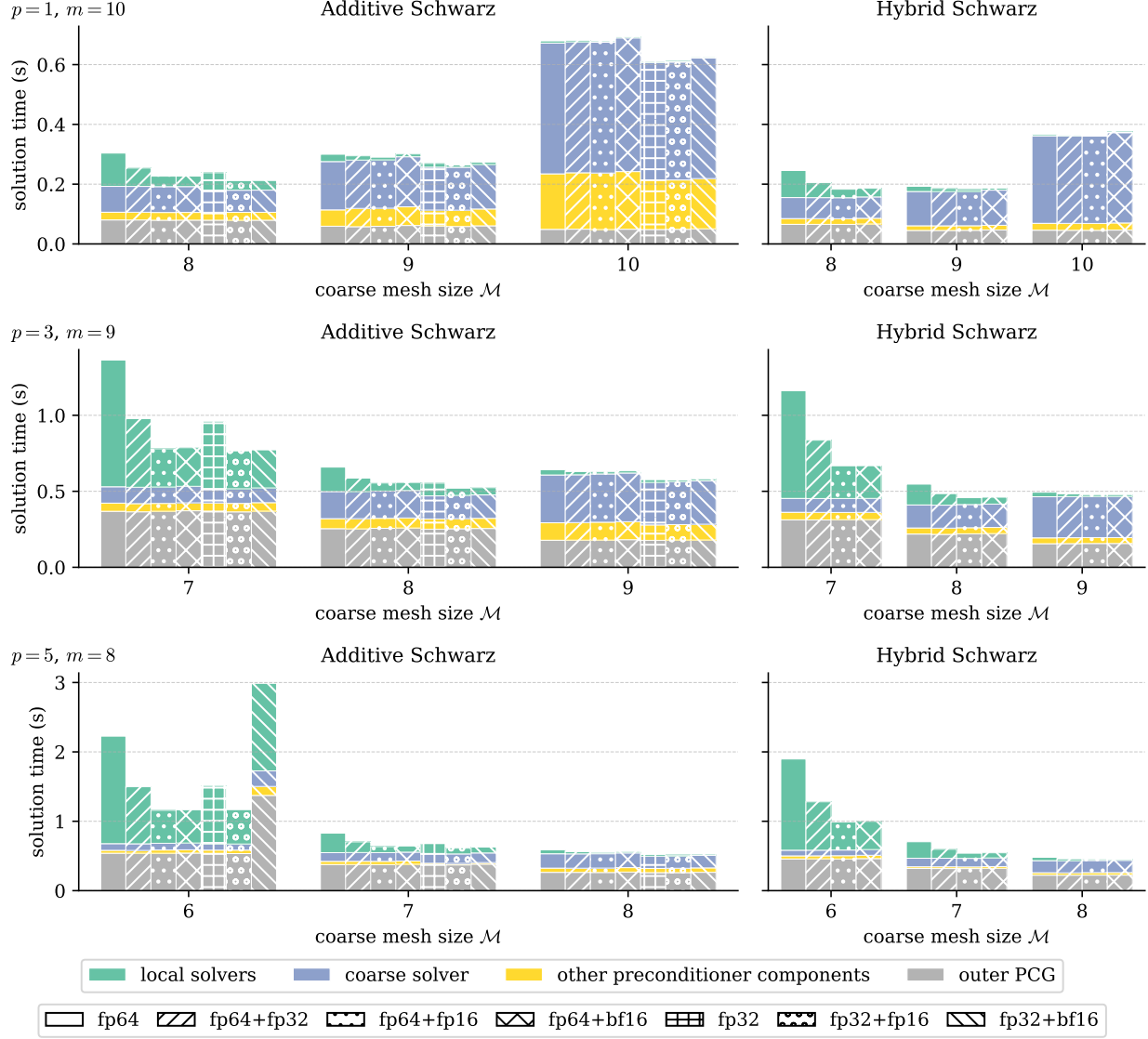


Figure 5.3. Component-wise solution times for various preconditioner precision settings. Single-precision variants of the hybrid operator are excluded because they fail to converge. Parameters: $d = 2$, $\mathcal{T}_h = \mathcal{S}_m$ and $\mathcal{T}_H = \mathcal{T}_h = \mathcal{S}_M$.

employs a sparse decomposition algorithm that relies on reading the integer indices of the nonzero entries, which are unaffected by floating-point precision changes. The remaining preconditioner components – application of R_C , R_C^T and A_R – are also indexed by integers; therefore their insensitivity to precision reduction is not surprising.

From these observations, we conclude that solving both local and coarse problems in reduced precision is beneficial, but should be used with care to avoid convergence issues. The largest performance gains may be achieved when the local solvers dominate the preconditioner cost; in all tested scenarios their precision can be safely set to **fp32** or **fp16** without significant degradation of convergence. Employing these settings results in speedups in the total solution time of up to $1.92\times$ when the mesh combination is fixed.

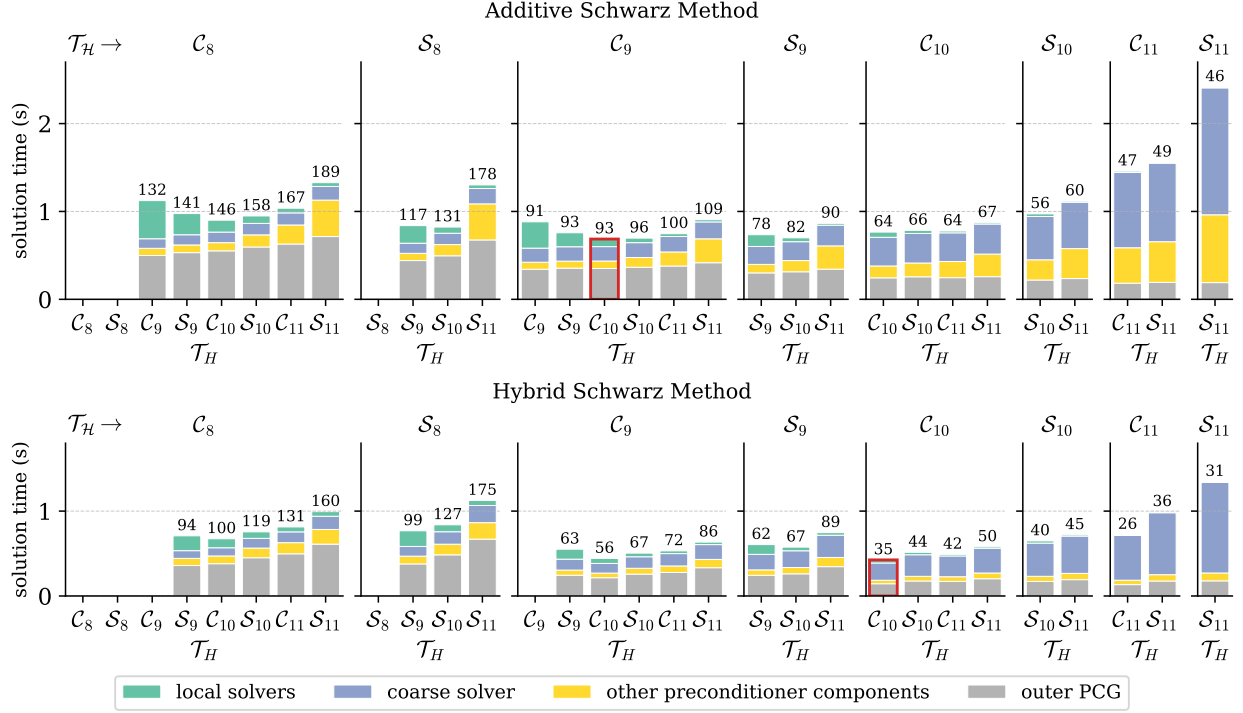


Figure 5.4. Component-wise solution times for various $\mathcal{T}_H$ and $\mathcal{T}_h$. Iteration counts are indicated above the bars, the best overall configuration for each preconditioner is highlighted in red. Parameters: $d = 2$, $p = 1$ and $\mathcal{T}_h = \mathcal{S}_{11}$.

5.3 Fine-tuning parallelism granularity

To achieve an optimal balance between parallelism and convergence speed, we investigate the performance of the additive and hybrid Schwarz preconditioners across various choices of the coarse mesh $\mathcal{T}_H$ and the local solvers' mesh $\mathcal{T}_h$. We set $d = 2$ and $p = 1$ to maximize the available parallelism: the local solvers' matrices can be as small as 3×3 when $\mathcal{T}_h = \mathcal{T}_H$. We fix $\mathcal{T}_h = \mathcal{S}_{11}$ as the fine mesh, which yields a discrete problem with roughly 25 MDoFs. Moreover, following the conclusions of the previous section, we employ mixed-precision settings: **fp32+fp16** and **fp64+fp16** for the additive and hybrid operators, respectively.

The results are shown in Figure 5.4. Missing entries indicate solver failures caused by out-of-memory errors, which occur when dense local problems exceed available GPU memory. The hybrid operator is more prone to this issue, as it requires storing the additional sparse matrix A_R (see Section 3).

5.3.1 Impact of mesh granularity on convergence rates

The measured number of iterations for both the additive and hybrid Schwarz preconditioners is consistent with the condition number bound (5.1). Minor deviations from monotonic growth with increasing M (for example, $\mathcal{T}_H = \mathcal{S}_{10}$ versus $\mathcal{T}_H = \mathcal{C}_{11}$ when $\mathcal{T}_h = \mathcal{C}_{10}$) can be attributed to the use of reduced precisions. As expected from Lemma 3.3, the hybrid operator consistently requires fewer iterations than the additive one. The largest reduction – up to a factor of $1.83\times$ – occurs when the local solvers' mesh is close to the coarse mesh.

5.3.2 Impact of mesh granularity on computational cost

As expected, the coarse solver dominates execution time when $\mathcal{T}_H \approx \mathcal{T}_h$, while local solvers cost peaks when $\mathcal{T}_H \approx \mathcal{T}_H$. Surprisingly, applying R_C and R_C^T in the additive operator becomes expensive when $\mathcal{T}_H \approx \mathcal{T}_h$, sometimes dominating the preconditioner cost. This overhead likely stems from the general `torch.segment_reduce` routine (see Listing 4.1), which supports an arbitrary distribution of elements across subdomains. This could be eliminated by assuming a constant number of DoFs per coarse element, but we opted to keep the implementation flexible. However, fusing the R_C application into the CSR matrix-vector product, as done in the hybrid method, substantially reduces this cost.

The above observations highlight the optimal balance between reducing the iteration count and limiting the preconditioner cost per iteration. With a fixed coarse mesh and $\mathcal{M} \leq 9$, the optimal $\mathcal{T}_H$ lies between $\mathcal{T}_H$ and $\mathcal{T}_h$, demonstrating the advantage of the three-grid Schwarz method. However, the best additive three-grid configuration is only $1.07\times$ faster than the best additive two-grid variant; moreover, the best hybrid configuration is already two-grid. We attribute this to the efficiency of cuDSS, which allows $\mathcal{T}_H$ to approach $\mathcal{T}_h$, leaving little room for improvement via an intermediate $\mathcal{T}_H$. Nevertheless, we expect the three-grid approach to offer greater potential on larger problems, which were excluded from this work due to single-GPU memory limitations.

The lower iteration counts of the hybrid preconditioner, combined with fusing the costly application of R_C into the CSR matrix-vector product, make it significantly faster than the additive variant, except in the few cases where the hybrid method fails due to out-of-memory errors. Speedups reach $2.01\times$ for fixed mesh combinations and $1.61\times$ when comparing the best overall configurations of both methods.

5.4 Comparison with AmgX

We conclude the experimental evaluation by comparing the performance of our solvers with the algebraic multigrid (AMG) preconditioners provided by AmgX [17, 18], a high-performance NVIDIA library for solving large sparse linear systems on NVIDIA GPUs. Because AmgX exposes many configurable parameters, careful tuning is required to obtain optimal performance for a given problem and, in our case, to ensure a fair comparison. For this reason, we give a detailed description of our AmgX setup before reporting the comparison results.

5.4.1 Selecting AmgX configuration

We evaluated all example configurations provided with AmgX (`AMGX/src/configs`), replacing GMRES and FGMRES solvers with PCG, and aligning the stopping criterion with the one employed in our experiments (see Section 5.1). The benchmarks used the system (2.3) obtained for $p = 1$, with $\mathcal{T}_h$ being a uniform simplicial mesh ($\mathcal{S}_m^d$) of a unit square ($d = 2$) or a unit cube ($d = 3$).

After filtering for successful convergence on small problems, we benchmarked the remaining candidates on medium and large problems. We retained only those that produced no runtime errors and were at most 50% slower than the fastest variant. This yielded four configurations: `AMG_CLASSICAL_L1_TRUNC`, `AMG_CLASSICAL_AGGRESSIVE_L1`, `AMG_CLASSICAL_AGGRESSIVE_L1_TRUNC` and `AMG_CLASSICAL_AGGRESSIVE_CHEB_L1_TRUNC`. From this point forward, we omit their common `AMG_CLASSICAL_` prefix for brevity. The selected configurations employ a classical algebraic multigrid preconditioner with a V-cycle and parallel coarsening algorithm (PMIS). We include all four variants in the performance comparison, reporting the best result for each test case. Note that we did not further tune AmgX settings here; additional performance gains may be possible through parameter optimization.

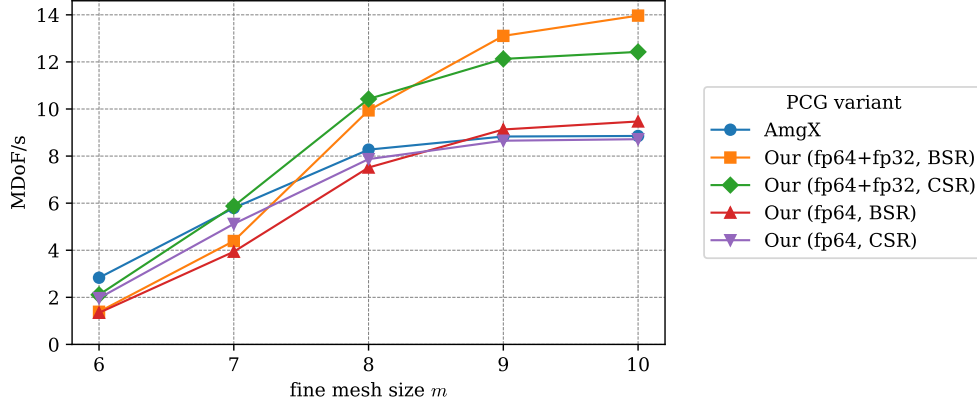


Figure 5.5. Throughput of the AmgX preconditioner L1_TRUNC across various PCG configurations. Parameters: $d = 2$, $p = 3$ and $\mathcal{T}_h = \mathcal{S}_m$.

AmgX can be used to build a complete solver, as it implements several Krylov methods, including PCG. However, it does not support mixed-precision computations or BSR matrix-vector product in combination with classical AMG. Furthermore, our primary interest is in comparing the preconditioners themselves rather than different PCG implementations. Therefore, we decided to accelerate AmgX in the same way as our Schwarz preconditioners – by combining it with our customized PCG implementation.

It turns out that this choice actually improves AmgX performance on large problems, as seen from the comparison of the L1_TRUNC preconditioner with the native AmgX PCG against various configurations of our PCG implementation. The results for $d = 2$ and $p = 3$ are shown in Figure 5.5. For small problems, our PCG is slightly slower due to PyTorch and Python overheads. However, as the problem size grows, these overheads become negligible, and the throughput of the AmgX PCG and our equivalent variant (fp64, CSR) becomes nearly identical. Moreover, enabling mixed precision and the BSR format yields further improvements, resulting in a $1.58\times$ speedup over the AmgX PCG.

5.4.2 Comparison results

For the final comparison, we consider $p = 1, \dots, 5$ for both $d = 2$ and $d = 3$. For each case, we test $\mathcal{T}_h \in \{\mathcal{S}_{m-1}, \mathcal{S}_m\}$, where $\mathcal{S}_m$ is the finest mesh yielding fewer than 2^{31} nonzeros in the matrix A . The additive and hybrid Schwarz operators use mixed-precision settings (fp32+fp16 and fp64+fp16, respectively), and for each test case, we select the best-performing configuration of $\mathcal{T}_h$ and $\mathcal{T}_H$ from the mesh families introduced in Section 5.1. Figure 5.6 presents the results in terms of solution throughput, defined as the number of unknowns divided by the solution time.

In 2D, while all AmgX configurations perform well for $p = 1$, only L1_TRUNC converges for $p > 1$, matching the performance of our Schwarz solvers. Although the additive variant trails AmgX, the hybrid method is consistently the fastest solver across all 2D problems. In 3D, AmgX performance drops substantially for $p \geq 3$. The best-performing configuration – mixed-precision L1_TRUNC – fails at $p = 4$, while its stable double-precision variant suffers performance degradation. In contrast, our Schwarz preconditioners maintain strong performance across all tested polynomial degrees, consistently achieving higher throughput than AmgX.

Details of the best-performing configurations of the hybrid operator are listed in Table 5.1. In all cases, it is optimal to choose $\mathcal{T}_h$ close to $\mathcal{T}_h$, which agrees with the conclusions of Section 5.3.

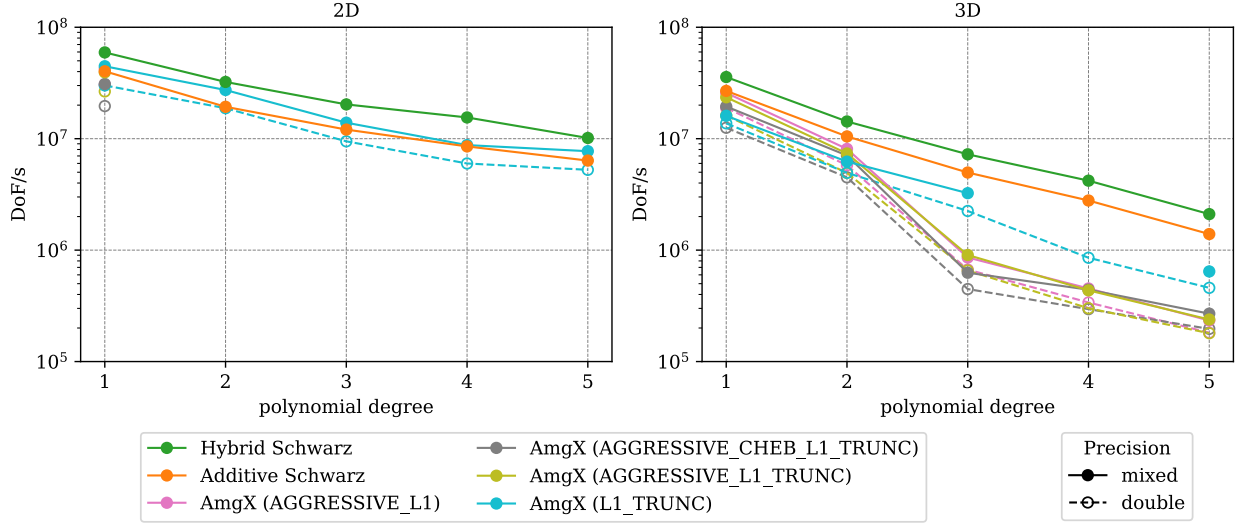


Figure 5.6. Maximum throughput (across tested $\mathcal{T}_h$) achieved by our Schwarz preconditioners and various AmgX variants on an A100 GPU.

Table 5.1. Best configurations of the hybrid Schwarz preconditioner and AmgX for each test case. For $d = 2$, $p = 4$, $\mathcal{T}_h = \mathcal{S}_{10}$, all AmgX configurations failed due to out-of-memory errors.

d	p	$\mathcal{T}_h$	MDoFs	best AmgX config found		best hybrid variant			
				name	time [s]	$\mathcal{T}_h$	$\mathcal{T}_H$	time [s]	speedup
2	1	$\mathcal{S}_{12}$	100.7	L1_TRUNC, fp32	2.25	$\mathcal{C}_{11}$	$\mathcal{C}_{12}$	1.82	1.23
	2	$\mathcal{S}_{11}$	50.3	L1_TRUNC, fp32	1.84	$\mathcal{C}_{10}$	$\mathcal{C}_{11}$	1.56	1.18
	3	$\mathcal{S}_{10}$	21.0	L1_TRUNC, fp32	1.51	$\mathcal{C}_{10}$	$\mathcal{C}_{10}$	1.03	1.46
	4	$\mathcal{S}_{10}$	31.5	—	—	$\mathcal{C}_{10}$	$\mathcal{C}_{10}$	2.03	∞
	5	$\mathcal{S}_9$	11.0	L1_TRUNC, fp32	1.43	$\mathcal{C}_9$	$\mathcal{C}_9$	1.08	1.31
3	1	$\mathcal{S}_7$	50.3	AGGRESSIVE_L1, fp32	1.95	$\mathcal{C}_6$	$\mathcal{C}_7$	1.41	1.39
	2	$\mathcal{S}_6$	15.7	AGGRESSIVE_L1, fp32	1.94	$\mathcal{C}_5$	$\mathcal{C}_6$	1.10	1.76
	3	$\mathcal{S}_5$	3.9	L1_TRUNC, fp32	1.21	$\mathcal{C}_5$	$\mathcal{C}_5$	0.54	2.23
	4	$\mathcal{S}_5$	6.9	L1_TRUNC, fp64	8.06	$\mathcal{C}_5$	$\mathcal{C}_5$	1.64	4.92
	5	$\mathcal{S}_4$	1.4	L1_TRUNC, fp32	2.14	$\mathcal{C}_4$	$\mathcal{C}_4$	0.65	3.27

Notably, the hybrid operator handles all considered problems successfully, demonstrating that even moderately large problems can be solved on a single GPU with a matrix-based approach.

6 Conclusions

The objective of this paper was to design, fine-tune and evaluate an efficient single-GPU implementation of the massively parallel domain decomposition preconditioner [11]. We adapted the original method to employ exact local solvers in mixed precision, extended it to support higher-degree finite elements and introduced a hybrid variant for improved performance. Our approach is based on PyTorch, enhanced with several targeted optimizations that may be of independent interest. These include a custom batched dense matrix-vector product kernel (10–70× faster than the PyTorch

baseline for small matrix sizes) and a BSR matrix-vector product algorithm that outperforms cuSPARSE by up to $2.68\times$ for small blocks.

Experiments showed that appropriately chosen mixed-precision configurations yield total time speedups of up to $1.92\times$ relative to double precision, with a negligible impact on the convergence rate. Furthermore, by introducing the hybrid Schwarz preconditioner, we improved the performance over the additive variant, reducing iteration counts by a factor of up to $1.83\times$ and achieving speedups of up to $1.61\times$. A comparison with NVIDIA’s AmgX demonstrated that our implementation is competitive: timings are comparable for 2D problems, while our method yields speedups up to $4.92\times$ over AmgX for high-degree 3D problems.

A parameter study revealed that single-GPU performance is maximized when local problems are kept very small (2–8 elements in 2D and 6 elements in 3D). This leverages the fine-grained parallelism of the local solvers and requires the coarse mesh to closely approximate the fine mesh. Consequently, a two-grid hierarchy is sufficient to achieve near-optimal performance. The three-grid variant seems better suited for larger problems exceeding single-GPU memory capacity.

We identify two main directions for future work. First, given the effectiveness of mixed precision, investigating inexact coarse solvers seems a promising avenue. Second, extending the approach to a multi-GPU setting [24] would enable larger problem sizes and is expected to better exploit the potential of the three-grid method.

Acknowledgements

The author thanks Dr. Piotr Krzyżanowski for many helpful discussions and valuable comments on the manuscript.

Code and data availability

The complete solver code and the obtained results are openly available in Zenodo at <https://doi.org/10.5281/zenodo.19850966>. The code was tested within the NVIDIA Docker container (Python 3.12.3, PyTorch 2.6.0 and NVIDIA CUDA 12.6.3), with all additional dependencies specified in the accompanying `Dockerfile`.

References

- [1] J. ANSEL, E. YANG, H. HE, N. GIMELSHEIN, A. JAIN, M. VOZNESENSKY, B. BAO, P. BELL, D. BERARD, E. BUROVSKI, G. CHAUHAN, A. CHOURDIA, W. CONSTABLE, A. DESMAISON, Z. DEVITO, E. ELLISON, W. FENG, J. GONG, M. GSCHWIND, B. HIRSH, S. HUANG, K. KALAMBARKAR, L. KIRSCH, M. LAZOS, M. LEZCANO, Y. LIANG, J. LIANG, Y. LU, C. K. LUK, B. MAHER, Y. PAN, C. PUHRSCHE, M. RESO, M. SAROUFIM, M. Y. SIRAICHI, H. SUK, S. ZHANG, M. SUO, P. TILLET, X. ZHAO, E. WANG, K. ZHOU, R. ZOU, X. WANG, A. MATHEWS, W. WEN, G. CHANAN, P. WU, AND S. CHINTALA, *PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation*, in Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS ’24, New York, NY, USA, 2024, Association for Computing Machinery, p. 929–947, <https://doi.org/10.1145/3620665.3640366>.
- [2] P. F. ANTONIETTI AND B. AYUSO, *Schwarz domain decomposition preconditioners for discontinuous Galerkin approximations of elliptic problems: non-overlapping case*, M2AN Math. Model. Numer. Anal., 41 (2007), pp. 21–54, <https://doi.org/10.1051/m2an:2007006>.

- [3] P. F. ANTONIETTI AND P. HOUSTON, *A class of domain decomposition preconditioners for hp-discontinuous Galerkin finite element methods*, J. Sci. Comput., 46 (2011), pp. 124–149, <https://doi.org/10.1007/s10915-010-9390-1>.
- [4] H. ANZT, J. DONGARRA, G. FLEGAR, N. J. HIGHAM, AND E. S. QUINTANA-ORTÍ, *Adaptive precision in block-Jacobi preconditioning for iterative sparse linear system solvers*, Concurrency and Computation: Practice and Experience, 31 (2019), p. e4460, <https://doi.org/10.1002/cpe.4460>.
- [5] I. A. BARATTA, J. P. DEAN, J. S. DOKKEN, M. HABERA, J. S. HALE, C. N. RICHARDSON, M. E. ROGNES, M. W. SCROGGS, N. SIME, AND G. N. WELLS, *DOLFINx: the next generation FEniCS problem solving environment*. Preprint, 2023, <https://doi.org/10.5281/zenodo.10447666>.
- [6] N. BELL, S. DALTON, AND L. N. OLSON, *Exposing fine-grained parallelism in algebraic multigrid methods*, SIAM J. Sci. Comput., 34 (2012), pp. C123–C152, <https://doi.org/10.1137/110838844>.
- [7] M. BERNASCHI, A. CELESTINI, F. VELLA, AND P. D’AMBRA, *A multi-GPU aggregation-based AMG preconditioner for iterative linear solvers*, IEEE Transactions on Parallel and Distributed Systems, 34 (2023), pp. 2365–2376, <https://doi.org/10.1109/TPDS.2023.3287238>.
- [8] Y. CHEN, X. TIAN, H. LIU, Z. CHEN, B. YANG, W. LIAO, P. ZHANG, R. HE, AND M. YANG, *Parallel ILU preconditioners in GPU computation*, Soft Comput., 22 (2018), p. 8187–8205, <https://doi.org/10.1007/s00500-017-2764-7>.
- [9] C. CUI AND G. KANSCHAT, *Multilevel interior penalty methods on GPUs*, ACM Trans. Math. Software, 51 (2025), pp. Art. 19, 27, <https://doi.org/10.1145/3765616>.
- [10] T. DEAKIN, J. PRICE, M. MARTINEAU, AND S. MCINTOSH-SMITH, *Evaluating attainable memory bandwidth of parallel programming models via BabelStream*, International Journal of Computational Science and Engineering, 17 (2018), pp. 247–262, <https://doi.org/10.1504/IJCSE.2018.095847>.
- [11] M. DRYJA AND P. KRZYŻANOWSKI, *A massively parallel nonoverlapping additive Schwarz method for discontinuous Galerkin discretization of elliptic problems*, Numer. Math., 132 (2016), pp. 347–367, <https://doi.org/10.1007/s00211-015-0718-5>.
- [12] R. EBERHARDT AND M. HOEMMEN, *Optimization of block sparse matrix-vector multiplication on shared-memory parallel architectures*, in 2016 IEEE International Parallel and Distributed Processing Symposium Workshops, IPDPS Workshops 2016, Chicago, IL, USA, May 23–27, 2016, IEEE Computer Society, 2016, pp. 663–672, <https://doi.org/10.1109/IPDPSW.2016.42>.
- [13] A. ERN AND J.-L. GUERMOND, *Theory and practice of finite elements*, vol. 159 of Applied Mathematical Sciences, Springer-Verlag, New York, 2004, <https://doi.org/10.1007/978-1-4757-4355-5>.
- [14] A. ERN, A. F. STEPHANSEN, AND P. ZUNINO, *A discontinuous Galerkin method with weighted averages for advection-diffusion equations with locally small and anisotropic diffusivity*, IMA J. Numer. Anal., 29 (2009), pp. 235–256, <https://doi.org/10.1093/imanum/drm050>.

- [15] P. KRZYŻANOWSKI, *On a nonoverlapping additive Schwarz method for h-p discontinuous Galerkin discretization of elliptic problems*, Numer. Methods Partial Differential Equations, 32 (2016), pp. 1572–1590, <https://doi.org/10.1002/num.22063>.
- [16] H. LIU, B. YANG, AND Z. CHEN, *Accelerating algebraic multigrid solvers on NVIDIA GPUs*, Comput. Math. Appl., 70 (2015), pp. 1162–1181, <https://doi.org/10.1016/j.camwa.2015.07.005>.
- [17] M. NAUMOV, M. ARSAEV, P. CASTONGUAY, J. COHEN, J. DEMOUTH, J. EATON, S. LAYTON, N. MARKOVSKIY, I. REGULY, N. SAKHARNYKH, V. SELLAPPAN, AND R. STRZODKA, *AmgX: a library for GPU accelerated algebraic multigrid and preconditioned iterative methods*, SIAM J. Sci. Comput., 37 (2015), pp. S602–S626, <https://doi.org/10.1137/140980260>.
- [18] NVIDIA, *AmgX reference manual*, 2017, https://raw.githubusercontent.com/NVIDIA/AMGX/main/doc/AMGX_Reference.pdf (accessed 2025-07-31).
- [19] NVIDIA, *CUDA C++ programming guide*, 2024, <https://docs.nvidia.com/cuda/archive/12.6.3/cuda-c-programming-guide/index.html> (accessed 2025-08-07).
- [20] NVIDIA, *cuSPARSE 12.6 API reference guide*, 2024, <https://docs.nvidia.com/cuda/archive/12.6.3/cusparse/index.html> (accessed 2025-07-26).
- [21] NVIDIA, *NVIDIA cuDSS (preview): A high-performance CUDA library for direct sparse solvers*, 2025, <https://docs.nvidia.com/cuda/cudss/index.html> (accessed 2025-07-27).
- [22] M. PAPADRAKAKIS, G. STAVROULAKIS, AND A. KARATARAKIS, *A new era in scientific computing: Domain decomposition methods in hybrid CPU–GPU architectures*, Computer Methods in Applied Mechanics and Engineering, 200 (2011), pp. 1490–1508, <https://doi.org/10.1016/j.cma.2011.01.013>.
- [23] A. TOSELLI AND O. WIDLUND, *Domain Decomposition Methods – Algorithms and Theory*, vol. 34, Springer Berlin, Heidelberg, 01 2005, <https://doi.org/10.1007/b137868>.
- [24] I. YAMAZAKI, A. HEINLEIN, AND S. RAJAMANICKAM, *An experimental study of two-level Schwarz domain-decomposition preconditioners on GPUs*, in 2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS), Los Alamitos, CA, USA, May 2023, IEEE Computer Society, pp. 680–689, <https://doi.org/10.1109/IPDPS54959.2023.00073>.
- [25] B. YANG, H. LIU, AND Z. CHEN, *Preconditioned GMRES solver on multiple-GPU architecture*, Comput. Math. Appl., 72 (2016), pp. 1076–1095, <https://doi.org/10.1016/j.camwa.2016.06.027>.
- [26] J. ŠÍSTEK AND T. OBERHUBER, *Acceleration of a parallel BDDC solver by using graphics processing units on subdomains*, The International Journal of High Performance Computing Applications, 37 (2023), pp. 151–164, <https://doi.org/10.1177/10943420221136873>.